

Редукция

Оптимизация параллельной редукции в CUDA



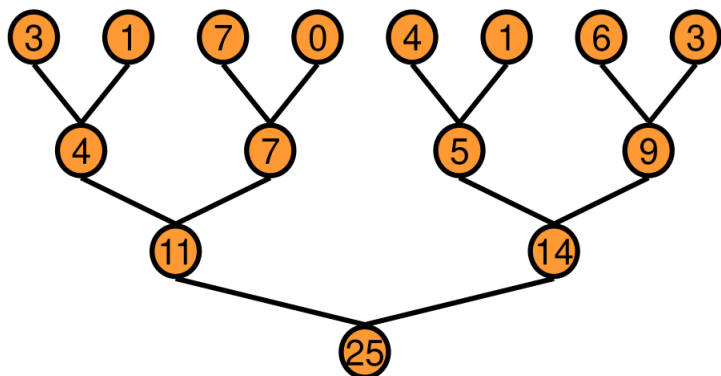
nVIDIA®

Марк Харрис
Технологии NVIDIA для разработчиков

Параллельная редукция

- Легко реализовать на CUDA, сложнее сделать это правильно
- Отличный пример оптимизации
- Пройдём шаг за шагом через 7 различных версий и продемонстрируем несколько важных стратегий оптимизации

Параллельная редукция



Древовидный подход, используемый в каждом блоке потока

Должна быть возможность использовать несколько блоков потоков: для обработки очень больших массивов и занятия всех мультипроцессоров на GPU. При этом каждый блок потока уменьшает часть массива.

Но **как мы обмениваемся** частичными результатами между блоками потоков?

Проблема: Глобальная синхронизация

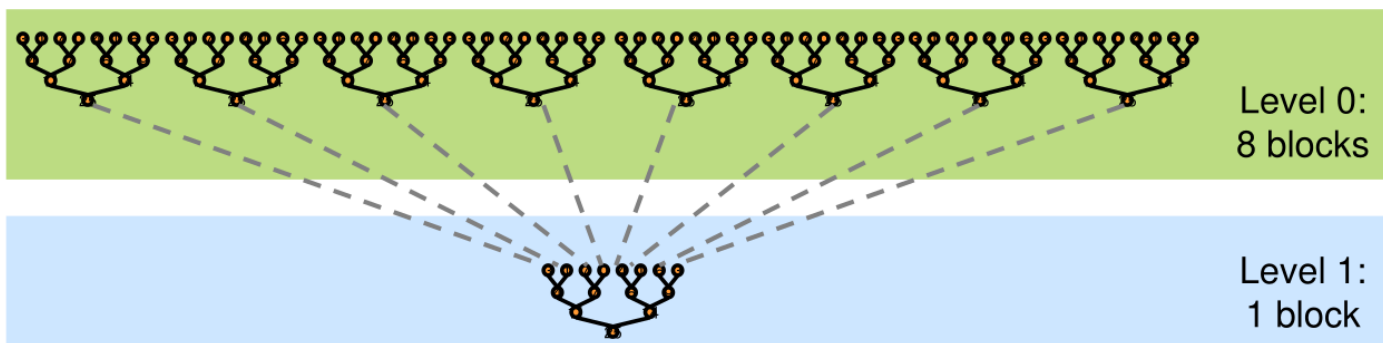
- Если бы могли синхронизировать все блоки потоков, мы могли бы легко уменьшить очень большие массивы, верно?
- Глобальная синхронизация после того, как каждый блок выдает свой результат и как только все блоки достигнут синхронизации, продолжаем рекурсивно.
- CUDA не имеет глобальной синхронизации. Почему?
 - Дорого встраивать аппаратное обеспечение для GPU с большим количеством процессоров.
 - Заставит программиста запускать меньше блоков (не более числа мультипроцессоров * число резидентных блоков или многопроцессор) для избежания тупика, который может снизить общую эффективность.

Проблема: Глобальная синхронизация

- Решение:
 - Запуск ядра служит точкой глобальной синхронизации и он имеет незначительные накладные расходы на аппаратное обеспечение.

Решение: Декомпозиция ядер

Избежание глобальной синхронизации,
разбивая вычисления на несколько вызовов
ядер.



В случае сокращений код для всех уровней
одинаков. Рекурсивный вызов ядра.

В чём цель оптимизации?

- Нам следует стремиться к максимальной производительности GPU.
- Выбираем правильную метрику:
 - Гфлопс/с: для вычислительных ядер.
 - Пропускная способность: для ядер с привязкой к памяти.

Сокращения имеют очень низкую арифметическую интенсивность. 1 флоп на каждый загруженный элемент и это оптимальная пропускная способность. Поэтому должны стремиться к максимальной пропускной способности.

- В этом примере используется G80 GPU
 - 384-битный интерфейс памяти, 900 МГц DDR
 - $384 * 1800 / 8 = 86.4$ ГБ/с

Редукция №1: Чередующая адресация

```
__global__ void reduce0(int *g_idata, int *g_odata) {  
    extern __shared__ int sdata[];  
    // каждый тред загружает один элемент из глобальной в общую  
    память  
    unsigned int tid = threadIdx.x;  
    unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;  
    sdata[tid] = g_idata[i];  
    __syncthreads();  
  
    // делаем редукцию в общей памяти  
    for(unsigned int s=1; s < blockDim.x; s *= 2) {  
        if (tid % (2*s) == 0) {  
            sdata[tid] += sdata[tid + s];  
        }  
        __syncthreads();  
    }  
  
    // пишем результат в этот блок к глобальной памяти  
    if (tid == 0) g_odata[blockIdx.x] = sdata[0];  
}
```

Чередующая адресация

Значения (общая память)

Шаг 1
Страйд 1

Тред
IDs

10	1	8	-1	0	-2	3	5	-2	-3	2	7	0	11	0	2
----	---	---	----	---	----	---	---	----	----	---	---	---	----	---	---



Значения

11	1	7	-1	-2	-2	8	5	-5	-3	9	7	11	11	2	2
----	---	---	----	----	----	---	---	----	----	---	---	----	----	---	---

Шаг 2
Страйд 2

Тред
IDs



Значения

18	1	7	-1	6	-2	8	5	4	-3	9	7	13	11	2	2
----	---	---	----	---	----	---	---	---	----	---	---	----	----	---	---

Шаг 3
Страйд 4

Тред
IDs



Значения

24	1	7	-1	6	-2	8	5	17	-3	9	7	13	11	2	2
----	---	---	----	---	----	---	---	----	----	---	---	----	----	---	---

Шаг 4
Страйд 8

Тред
IDs



Значения

41	1	7	-1	6	-2	8	5	17	-3	9	7	13	11	2	2
----	---	---	----	---	----	---	---	----	----	---	---	----	----	---	---

Редукция №1: Чередующая адресация

```
__global__ void reduce1(int *g_idata, int *g_odata) {
    extern __shared__ int sdata[];

    // каждый тред загружает один элемент из глобальной в общую память
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
    sdata[tid] = g_idata[i];
    __syncthreads();
    // делаем редукцию в общей памяти
    for (unsigned int s=1; s < blockDim.x; s *= 2) {

        if (tid % (2*s) == 0) { // Проблема: сильно расходящиеся варпы очень
неэффективны, а оператор % очень медленный
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    // пишем результат в этот блок к глобальной памяти
    if (tid == 0) g_odata[blockIdx.x] = sdata[0];
}
```

Производительность для уменьшения количества 4M

	Время (2^{22} ints)	Пропускная способность
Ядро 1: Чередующаяся адресация с расходящимся ветвлением	8.054 мс	2.083 ГБ/с

Примечание: Размер блока = 128 потоков для всех тестов.

Редукция №2: Чередующая адресация

Просто замените расходящуюся ветвь во
внутреннем цикле:

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {  
    if (tid % (2*s) == 0) {  
        sdata[tid] += sdata[tid + s];  
    }  
    __syncthreads();  
}
```

С шаговым индексом и нерасходящейся ветвью:

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {  
    int index = 2 * s * tid;  
    if (index < blockDim.x) {  
        sdata[index] += sdata[index + s];  
    }  
    __syncthreads();  
}
```

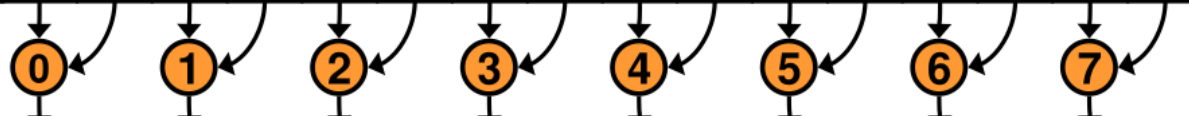
Параллельная редукция: Чередующая адресация

Значения (общая память)

Шаг 1
Страйд 1

Тред
IDs

10	1	8	-1	0	-2	3	5	-2	-3	2	7	0	11	0	2
----	---	---	----	---	----	---	---	----	----	---	---	---	----	---	---



Значения

Шаг 2
Страйд 2

Тред
IDs

11	1	7	-1	-2	-2	8	5	-5	-3	9	7	11	11	2	2
----	---	---	----	----	----	---	---	----	----	---	---	----	----	---	---

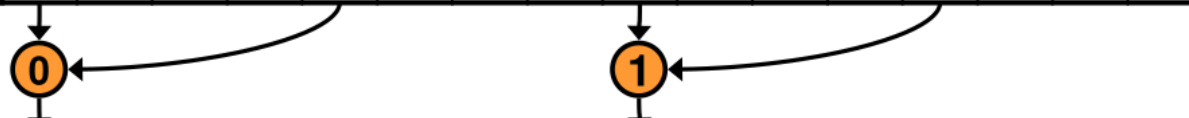


Значения

Шаг 3
Страйд 4

Тред
IDs

18	1	7	-1	6	-2	8	5	4	-3	9	7	13	11	2	2
----	---	---	----	---	----	---	---	---	----	---	---	----	----	---	---



Значения

Шаг 4
Страйд 8

Тред
IDs

24	1	7	-1	6	-2	8	5	17	-3	9	7	13	11	2	2
----	---	---	----	---	----	---	---	----	----	---	---	----	----	---	---



Значения

41	1	7	-1	6	-2	8	5	17	-3	9	7	13	11	2	2
----	---	---	----	---	----	---	---	----	----	---	---	----	----	---	---

Новая проблема: конфликты общих банков памяти

Производительность для уменьшения количества 4М

	Время (2 ²² ints)	Пропускная способность	Шаговое ускорение	Суммарное ускорение
Ядро 1: Чередующаяся адресация с расходящимся ветвлением	8.054 мс	2.083 ГБ/с		
Ядро 2: чередующаяся адресация с расходящимся ветвлением с конфликтами общих банков памяти	3.456 мс	4.854 ГБ/с	2.33	2.33

Параллельная редукция: последовательная адресация

Значения (общая
память)

10	1	8	-1	0	-2	3	5	-2	-3	2	7	0	11	0	2
----	---	---	----	---	----	---	---	----	----	---	---	---	----	---	---

Шаг 1

Страйд 1

Тред
IDs

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

Шаг 2

Страйд 2

Значения

8	-2	10	6	0	9	3	7	-2	-3	2	7	0	11	0	2
---	----	----	---	---	---	---	---	----	----	---	---	---	----	---	---

Тред
IDs

0	1	2	3
---	---	---	---

Шаг 3

Страйд 4

Значения

8	7	13	13	0	9	3	7	-2	-3	2	7	0	11	0	2
---	---	----	----	---	---	---	---	----	----	---	---	---	----	---	---

Тред
IDs

0	1
---	---

Шаг 4

Страйд 8

Значения

21	20	13	13	0	9	3	7	-2	-3	2	7	0	11	0	2
----	----	----	----	---	---	---	---	----	----	---	---	---	----	---	---

Тред
IDs

0

Значения

41	20	13	13	0	9	3	7	-2	-3	2	7	0	11	0	2
----	----	----	----	---	---	---	---	----	----	---	---	---	----	---	---

Последовательная адресация бесконфликтна

Редукция №3: Последовательная адресация

Просто замените шаговую индексацию во внутреннем цикле:

```
for (unsigned int s=1; s < blockDim.x; s *= 2) {  
    int index = 2 * s * tid;  
    if (index < blockDim.x) {  
        sdata[index] += sdata[index + s];  
    }  
    __syncthreads();  
}
```

С обратным циклом и индексацией на основе тредID:

```
for (unsigned int s=blockDim.x/2; s>0; s>>=1) {  
    if (tid < s) {  
        sdata[tid] += sdata[tid + s];  
    }  
    __syncthreads();  
}
```

Производительность для уменьшения количества 4М

	Время (2 ²² ints)	Пропускная способность	Шаговое ускорение	Суммарное ускорение
Ядро 1: Чередующаяся адресация с расходящимся ветвлением	8.054 мс	2.083 ГБ/с		
Ядро 2: чередующаяся адресация с конфликтами общих банков памяти	3.456 мс	4.854 ГБ/с	2.33	2.33
Ядро 3: Последовательная адресация	1.722 мс	9.741 ГБ/с	2.01	4.68

Бездействующие потоки

Проблема:

```
for (unsigned int s=blockDim.x/2; s>0; s>>=1) {  
    if (tid < s) {  
        sdata[tid] += sdata[tid + s];  
    }  
    __syncthreads();  
}
```

Половина потоков простаивает на первой итерации
цикла!
Это расточительно...

Редукция № 4: первое добавление во время загрузки

Уменьшите вдвое количество блоков и замените
одиночную загрузку:

// каждый поток загружает один элемент из глобальной в общую
память

```
unsigned int tid = threadIdx.x;  
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;  
sdata[tid] = g_idata
```

С двумя нагрузками и первым сложением
уменьшения:

```
// выполнить первый уровень редукции,  
// чтение из глобальной памяти, запись в разделяемую память  
unsigned int tid = threadIdx.x;  
unsigned int i = blockIdx.x*(blockDim.x*2) + threadIdx.x;  
sdata[tid] = g_idata[i] + g_idata[i+blockDim.x];  
__syncthreads();
```

Производительность для уменьшения количества 4М

	Время (2 ²² ints)	Пропускная способность	Шаговое ускорение	Суммарное ускорение
Ядро 1: чередующаяся адресация с расходящимся ветвлением	8.054 мс	2.083 ГБ/с		
Ядро 2: чередующаяся адресация с с конфликтами общих банков памяти	3.456 мс	4.854 ГБ/с	2.33	2.33
Ядро 3: Последовательная адресация	1.722 мс	9.741 ГБ/с	2.01	4.68
Ядро 4: первое добавление во время глобальной загрузки	0.965 мс	17.377 ГБ/с	1.78	8.34

Узкое место инструкции

- При скорости 17 ГБ/с мы далеки от ограничения пропускной способности и мы знаем, что сокращение имеет низкую арифметическую интенсивность.
- Поэтому вероятным узким местом являются накладные расходы на инструкции:
 - Вспомогательные инструкции, которые не являются загрузкой, сохранением или арифметикой для основных вычислений.
 - Другими словами: адресная арифметика и накладные расходы цикла.
- Стратегия: развернуть петли.

Развертывание последней деформации

- По мере сокращения количество «активных» потоков уменьшается.
 - Когда $s \leq 32$, у нас остается только один варп.
- Инструкции являются SIMD-синхронными внутри варпа.
- Это означает, что когда $s \leq 32$:
 - Нам не нужно `__syncthreads()`
 - Нам не нужно «`if (tid < s)`», потому что это не экономит работу.
- Давайте развернем последние 6 итераций внутреннего цикла.

Редукция №5: развернём последнюю деформацию

__device__ void warpReduce(volatile int* sdata, int tid) { // ВАЖНО:

чтобы это было правильно, мы должны использовать ключевое слово

«volatile»

```
sdata[tid] += sdata[tid + 32];  
sdata[tid] += sdata[tid + 16];  
sdata[tid] += sdata[tid + 8];  
sdata[tid] += sdata[tid + 4];  
sdata[tid] += sdata[tid + 2];  
sdata[tid] += sdata[tid + 1];  
}
```

// позже...

```
for (unsigned int s=blockDim.x/2; s>32; s>>=1) {  
    if (tid < s)  
        sdata[tid] += sdata[tid + s];  
    __syncthreads();  
}
```

if (tid < 32) warpReduce(sdata, tid);

Примечание: Это избавляет от бесполезной работы
во всех деформациях, а не только в последней!
Без развертывания все варпы выполняют каждую
итерацию цикла for и оператора if.

Производительность для уменьшения количества 4М

	Время (2 ²² ints)	Пропускная способность	Шаговое ускорение	Суммарное ускорение
Ядро 1: чередующаяся адресация с расходящимся ветвлением	8.054 мс	2.083 ГБ/с		
Ядро 2: чередующаяся адресация с с конфликтами общих банков памяти	3.456 мс	4.854 ГБ/с	2.33	2.33
Ядро 3: Последовательная адресация	1.722 мс	9.741 ГБ/с	2.01	4.68
Ядро 4: первое добавление во время глобальной загрузки	0.965 мс	17.377 ГБ/с	1.78	8.34
Ядро 5: Развернули последнюю деформацию	0.536 мс	31.289 ГБ/с	1.8	15.01

Полное развертывание

- Если бы мы знали количество итераций во время компиляции, мы могли бы полностью развернуть сокращение
 - К счастью, размер блока ограничен GPU до 512 потоков. Кроме того, мы придерживаемся размеров блоков, равных степени 2.
- Таким образом, мы можем легко развернуться для фиксированного размера блока, но нам нужно быть универсальными — как мы можем развернуться для размеров блоков, которые мы не знаем во время компиляции?
- Шаблоны в помощь! CUDA поддерживает параметры шаблона C++ на устройстве и хост-функции.

Развертывание с помощью шаблонов

Укажите размер блока в качестве параметра
шаблона функции:

```
template <unsigned int blockSize>  
__global__ void reduce5(int *g_idata, int *g_odata)
```

Редукция № 6: Полностью развернутый

Template <unsigned int blockSize>

```
__device__ void warpReduce(volatile int* sdata, int tid) {  
    if (blockSize >= 64) sdata[tid] += sdata[tid + 32];  
    if (blockSize >= 32) sdata[tid] += sdata[tid + 16];  
    if (blockSize >= 16) sdata[tid] += sdata[tid + 8];  
    if (blockSize >= 8) sdata[tid] += sdata[tid + 4];  
    if (blockSize >= 4) sdata[tid] += sdata[tid + 2];  
    if (blockSize >= 2) sdata[tid] += sdata[tid + 1];  
}
```

```
    if (blockSize >= 512) {  
        if (tid < 256) { sdata[tid] += sdata[tid + 256]; } __syncthreads(); }  
    if (blockSize >= 256) {  
        if (tid < 128) { sdata[tid] += sdata[tid + 128]; } __syncthreads(); }  
    if (blockSize >= 128) {  
        if (tid < 64) { sdata[tid] += sdata[tid + 64]; } __syncthreads(); }  
}
```

```
if (tid < 32) warpReduce<blockSize>(sdata, tid);
```

Примечание: весь **синий** код будет оцениваться во время компиляции.

В результате получается очень эффективный внутренний цикл!

Вызов ядра шаблона

Разве нам не нужен размер блока во время компиляции? Нет, просто оператор switch для 10 возможных размеров блока:

```
switch (threads)
{
  case 512:
    reduce5<512><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 256:
    reduce5<256><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 128:
    reduce5<128><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 64:
    reduce5< 64><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 32:
    reduce5< 32><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 16:
    reduce5< 16><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 8:
    reduce5< 8><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 4:
    reduce5< 4><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 2:
    reduce5< 2><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
  case 1:
    reduce5< 1><<< dimGrid, dimBlock, smemSize >>>(d_idata, d_odata); break;
}
```

Производительность для уменьшения количества 4М

	Время (2 ²² ints)	Пропускная способность	Шаговое ускорение	Суммарное ускорение
Ядро 1: чередующаяся адресация с расходящимся ветвлением	8.054 мс	2.083 ГБ/с		
Ядро 2: чередующаяся адресация с с конфликтами общих банков памяти	3.456 мс	4.854 ГБ/с	2.33	2.33
Ядро 3: Последовательная адресация	1.722 мс	9.741 ГБ/с	2.01	4.68
Ядро 4: первое добавление во время глобальной загрузки	0.965 мс	17.377 ГБ/с	1.78	8.34
Ядро 5: Развернули последнюю деформацию	0.536 мс	31.289 ГБ/с	1.8	15.01
Ядро 6: полностью развёрнутый	0.381 мс	43.996 ГБ/с	1.41	21.16

Параллельная редукция сложности

- $\log(N)$ параллельные шаги, каждый шаг S делает $N/2^S$ независимых операций
 - Сложность шага составляет $O(\log N)$
- Для $N=2^D$ выполняет $\sum_{S \in [1..D]} 2^{D-S} = N-1$ операций
 - Сложность работы $O(N)$ — это эффективно. То есть не выполняет больше операций, чем последовательный алгоритм
- При физически параллельных потоках P (процессоры P) временная сложность составляет $O(N/P + \log N)$.
 - Сравните с $O(N)$ для последовательного сокращения.
 - В блоке потоков $N=P$, поэтому $O(\log N)$

Как насчет стоимости?

- Стоимость параллельного алгоритма - временная сложность процессоров
 - Выделять потоки вместо процессоров: $O(N)$ потоков.
 - Временная сложность составляет $O(\log N)$, поэтому стоимость $O(N \log N)$: **нерентабельно!**
- Теорема Брента предполагает $O(N/\log N)$ потоков
 - Каждый поток выполняет последовательную работу $O(\log N)$. Тогда все потоки $O(N/\log N)$ взаимодействуют в течение $O(\log N)$ шагов. Стоимость = $O((N/\log N) * \log N) = O(N) \rightarrow$ экономичность
- Иногда называется каскадным алгоритмом. Может привести к значительному ускорению на практике.

Каскадный алгоритм

- Комбинируйте последовательное и параллельное сокращение
 - Каждый поток загружает и суммирует несколько элементов в разделяемую память. Сокращение разделяемой памяти на основе дерева
- Теорема Брента говорит, что каждая нить должна суммировать $O(\log n)$ элементов, то есть 1024 или 2048 элементов на блок против 256.
- По опыту, полезно продвигать его еще дальше.
 - Возможно лучшее сокращение задержки за счет увеличения объема работы на поток.
 - Больше потоков на блок снижает количество уровней в дереве рекурсивных вызовов ядра.
 - Высокие накладные расходы на запуск ядра на последних уровнях с несколькими блоками

Каскадный алгоритм

- На G80 лучшая производительность с 64-256 блоками по 128 потоков
 - 1024-4096 элементов на поток

Редукция № 7: несколько добавлений/потоков

Заменить нагрузку и добавить два элемента:

```
unsigned int tid = threadIdx.x;
unsigned int i = blockIdx.x*(blockDim.x*2) + threadIdx.x;
sdata[tid] = g_idata[i] + g_idata[i+blockDim.x];
__syncthreads();
```

С циклом while, чтобы добавить столько, сколько необходимо:

```
unsigned int tid = threadIdx.x;
unsigned int i = blockIdx.x*(blockSize*2) + threadIdx.x;
unsigned int gridSize = blockSize*2*gridDim.x;
sdata[tid] = 0;
while (i < n) {
    sdata[tid] += g_idata[i] + g_idata[i+blockSize];
    i += gridSize;
}
__syncthreads();
```

Редукция № 7: несколько добавлений/потоков

Заменить нагрузку и добавить два элемента:

```
unsigned int tid = threadIdx.x;
unsigned int i = blockIdx.x*(blockDim.x*2) + threadIdx.x;
sdata[tid] = g_idata[i] + g_idata[i+blockDim.x];
__syncthreads();
```

С циклом while, чтобы добавить столько, сколько необходимо:

```
unsigned int tid = threadIdx.x;
unsigned int i = blockIdx.x*(blockSize*2) + threadIdx.x;
unsigned int gridSize = blockSize*2*gridDim.x;
sdata[tid] = 0;
while (i < n) {
    sdata[tid] += g_idata[i] + g_idata[i+blockSize];
    i += gridSize; //Примечание: шаг петли gridSize для сохранения
объединения!
}
__syncthreads();
```

Производительность для уменьшения количества 4М

	Время (2 ²² ints)	Пропускная способность	Шаговое ускорение	Суммарное ускорение
Ядро 1: чередующаяся адресация с расходящимся ветвлением	8.054 мс	2.083 ГБ/с		
Ядро 2: чередующаяся адресация с с конфликтами общих банков памяти	3.456 мс	4.854 ГБ/с	2.33	2.33
Ядро 3: Последовательная адресация	1.722 мс	9.741 ГБ/с	2.01	4.68
Ядро 4: первое добавление во время глобальной загрузки	0.965 мс	17.377 ГБ/с	1.78	8.34
Ядро 5: Развернули последнюю деформацию	0.536 мс	31.289 ГБ/с	1.8	15.01
Ядро 6: полностью развёрнутый	0.381 мс	43.996 ГБ/с	1.41	21.16
Ядро 7: полностью развёрнутый	0.268 мс	62.671 ГБ/с	1.42	30.04

Ядро 7 на 32 млн элементов: **73 ГБ/с!**

Окончательное оптимизированное ядро

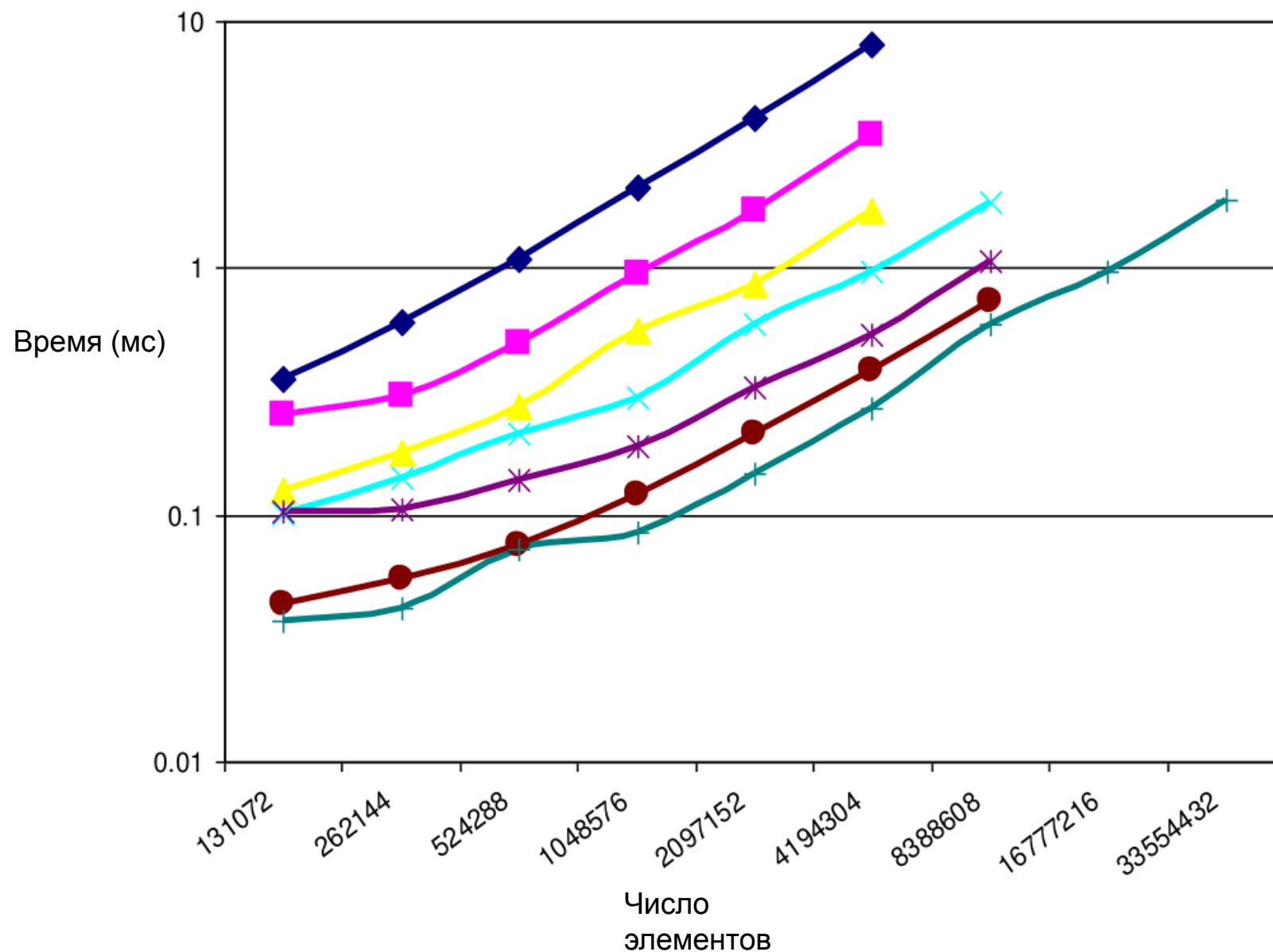
```
template <unsigned int blockSize>
__device__ void warpReduce(volatile int *sdata, unsigned int tid) {
    if (blockSize >= 64) sdata[tid] += sdata[tid + 32];
    if (blockSize >= 32) sdata[tid] += sdata[tid + 16];
    if (blockSize >= 16) sdata[tid] += sdata[tid + 8];
    if (blockSize >= 8) sdata[tid] += sdata[tid + 4];
    if (blockSize >= 4) sdata[tid] += sdata[tid + 2];
    if (blockSize >= 2) sdata[tid] += sdata[tid + 1];
}

template <unsigned int blockSize>
__global__ void reduce6(int *g_idata, int *g_odata, unsigned int n) {
    extern __shared__ int sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x*(blockSize*2) + tid;
    unsigned int gridSize = blockSize*2*gridDim.x;
    sdata[tid] = 0;
    while (i < n) { sdata[tid] += g_idata[i] + g_idata[i+blockSize]; i += gridSize; }
    __syncthreads();

    if (blockSize >= 512) { if (tid < 256) { sdata[tid] += sdata[tid + 256]; } __syncthreads(); }
    if (blockSize >= 256) { if (tid < 128) { sdata[tid] += sdata[tid + 128]; } __syncthreads(); }
    if (blockSize >= 128) { if (tid < 64) { sdata[tid] += sdata[tid + 64]; } __syncthreads(); }

    if (tid < 32) warpReduce(sdata, tid);
    if (tid == 0) g_odata[blockIdx.x] = sdata[0];
}
```

Сравнение производительности



- 1: чередующаяся адресация: Расходящиеся ветви
- 2: Чередующаяся адресация: конфликты банков
- 3: Последовательная адресация
- 4: Первое добавление во время глобальной загрузки
- 5: Развернуть последний варп
- 6: полностью развернуть
- 7: Несколько элементов на поток (максимум 64 блока)

Типы оптимизации

- Интересное наблюдение:
- Алгоритмическая оптимизация
 - Изменения в адресации, каскадирование алгоритмов.
 - 11,84-кратное ускорение в сумме!
- Оптимизация кода
 - Развертывание цикла
 - 2.54-кратное ускорение, комбинированное

Заключение

- Понимание характеристик производительности CUDA
 - Объединение памяти.
 - Расходящееся ветвление
 - Банковские конфликты
 - Скрытие задержки
- Используйте показатели максимальной производительности для оптимизации
- Понимание теории сложности параллельных алгоритмов
- Знать, как определить тип узкого места
 - например память, основные вычисления или служебные инструкции
- Оптимизируйте свой алгоритм, затем разверните циклы
- Используйте параметры шаблона для создания оптимального кода
- Вопросы: mharris@nvidia.com

Редукция